

ДИСЦИПЛИНА	Алгоритмы и структуры данных
ИНСТИТУТ	Институт перспективных технологий и индустриального программирования
КАФЕДРА	Кафедра индустриального программирования
ВИД УЧЕБНОГО МАТЕРИАЛА	Текущий контроль
ПРЕПОДАВАТЕЛЬ	Преснецова Виктория Юрьевна, Яковлев Михаил Сергеевич, Дворецкий Артур Геннадьевич, Гиматдинов Дамир Маратович
СЕМЕСТР	2 семестр, 2024-2025 гг.

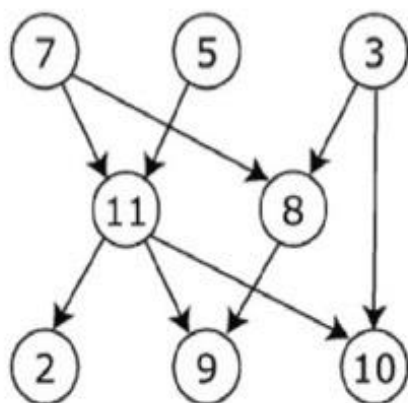
Рабочая тетрадь 5.
Алгоритмы на графах

Требования
1. Код должен быть оптимизирован для производительности и использования ресурсов. 2. Необходимо избегать избыточных вычислений и памяти. 3. Комментарии должны объяснять сложные участки кода и логику работы программы.

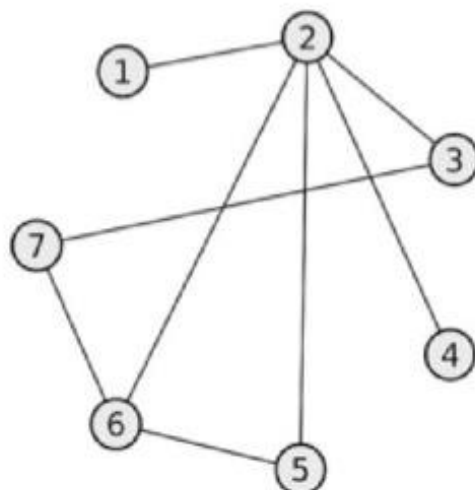
Основные понятия теории графов

Теоретический материал
<i>Граф</i> – математический объект, состоящий из двух множеств. Одно из них – любое конечное множество, его элементы называются <i>вершинами</i> графа. Другое множество состоит из пар вершин, эти пары называются <i>ребрами</i> графа. Если множество вершин графа обозначено буквой <i>V</i>, множество ребер – буквой <i>E</i>, а сам граф – буквой <i>G</i>, то пишут $G = (V, E)$ Если ребра – упорядоченные пары, то такой граф называется <i>ориентированным</i> (сокращенно <i>орграф</i>), если же неупорядоченные, то <i>неориентированным</i>.

ориентированный



неориентированный



Говорят, что ребро $e = (a,b)$ *инцидентно* каждой из вершин a и b , а каждая из этих вершин *инцидентна* ребру e .

Множество вершин, смежных с данной вершиной x в некотором графе, называется *окрестностью* этой вершины и обозначается через $N(x)$. Число вершин в $N(x)$ называется *степенью* вершины x и обозначается через $\deg(x)$, т.е.

$$\deg(x) = |N(x)|$$

Для графа с m ребрами и множеством вершин V выполняется равенство

$$\sum_{x \in V} \deg(x) = 2m.$$

В ориентированном графе для каждой вершины $\deg^+(x)$ можно ввести два числа:

$\deg^+(x)$ — число входящих ребер, т.е. ребер вида (y,x)

и

$\deg^-(x)$ — число выходящих, т.е. ребер вида (x,y) .

Эти числа называют соответственно степенью захода и степенью исхода вершины. Аналогом теоремы о рукопожатиях для ориентированного графа является очевидное равенство

$$\sum_{x \in V} \deg^+(x) = \sum_{x \in V} \deg^-(x).$$

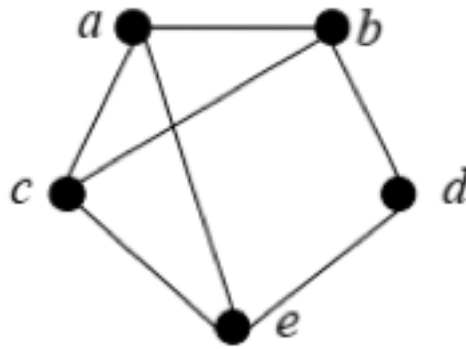
Способы представления графа:

1. Перечисление элементов. Для того, чтобы задать граф, достаточно перечислить его вершины и ребра (т.е. пары вершин). Пример:

$$V = \{a, b, c, d, e\}$$

$$E = \{(a, b), (a, c), (a, e), (b, c), (b, d), (c, e), (d, e)\}$$

2. Изображение. Если граф не слишком большой, его можно нарисовать. В неориентированном графе ребра изображают линиями, соединяющими смежные вершины, в ориентированном — стрелками.



3. Матрица смежности. Пусть G – граф с n вершинами, пронумерованными числами от 1 до n . Матрица смежности – это таблица с n строками и n столбцами, в которой элемент, стоящий на пересечении строки с номером i и столбца с номером j , равен 1, если вершины с номерами i и j смежны, и 0, если они не смежны.

Пример (вершины пронумерованы в алфавитном порядке):

$$\begin{pmatrix} 0 & 1 & 1 & 0 & 1 \\ 1 & 0 & 1 & 1 & 0 \\ 1 & 1 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 & 1 \\ 1 & 0 & 1 & 1 & 0 \end{pmatrix}$$

4. Матрица инцидентности. Пусть G – граф, вершины которого пронумерованы числами от 1 до n , а ребра – числами от 1 до m . В матрице инцидентности строки соответствуют вершинам, а столбцы – ребрам. На пересечении строки с номером i и столбца с номером j стоит 1, если вершина с номером i инцидентна ребру с номером j , и 0 в противном случае.

Пример:

$$\begin{pmatrix} 1 & 1 & 1 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 1 & 1 & 0 & 0 \\ 0 & 1 & 0 & 1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 & 0 & 1 & 1 \end{pmatrix}$$

5. Списки смежности. Этот способ часто используется для компьютерного представления графов. Состоит он в том, что для каждой вершины задается список всех смежных с ней вершин. В структурах данных, применяемых в программировании, списки смежности могут быть реализованы как массив линейных списков. В формулировках задач будем эти списки оформлять так: пишется номер или имя вершины и после двоеточия перечисляются все смежные с ней вершины. Пример:

- 1: 2, 3, 5;
- 2: 1, 3, 4;
- 3: 1, 2, 5;
- 4: 2, 5;
- 5: 1, 3, 4.

Маршрут– это последовательность вершин и ребер **графа**, следуя по которым, можно попасть из одной его вершины в другую.

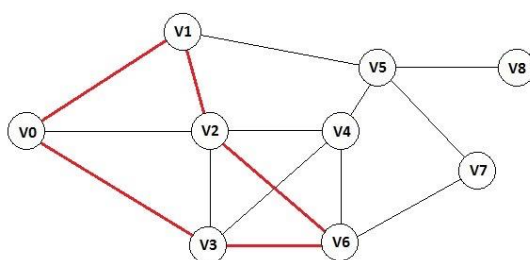
Цепь – это **маршрут** без повторяющихся ребер.

Путь в графе – это последовательность вершин $x_1, x_2, \dots, x_k$, в которой каждая пара (x_i, x_{i+1}) , $i = 1, 2, \dots, k - 1$, является ребром, причем все эти ребра различны. Путь *соединяет* вершины x_1 и x_k . *Длиной пути* называется число ребер в нем, т.е. $k - 1$.

Цикл – путь, у которого $x_1 = x_k$.

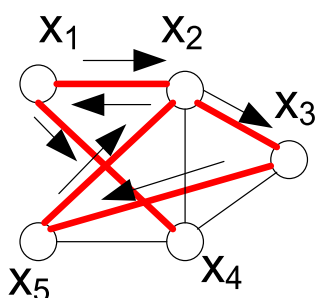
Цикл называется *простым*, если все его ребра различны, и *элементарным*, если все вершины, через которые он проходит, *различны*.

Например: $V_0-V_1-V_2-V_6-V_3-V_0$

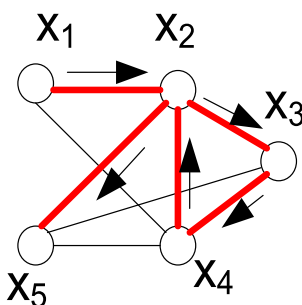


Цепи и циклы будут *простыми*, если они не содержат повторяющихся вершин, например, x_1, x_2, x_3, x_4, x_5 – простая цепь, а x_1, x_2, x_3, x_4, x_1 – простой цикл.

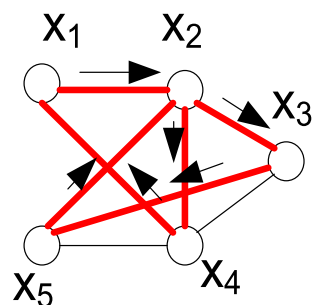
Маршрут
 $x_1, x_2, x_3, x_5, x_2, x_1, x_4$



Цепь
 $x_1, x_2, x_3, x_4, x_2, x_5$



Цикл
 $x_1, x_2, x_3, x_5, x_2, x_4, x_1$

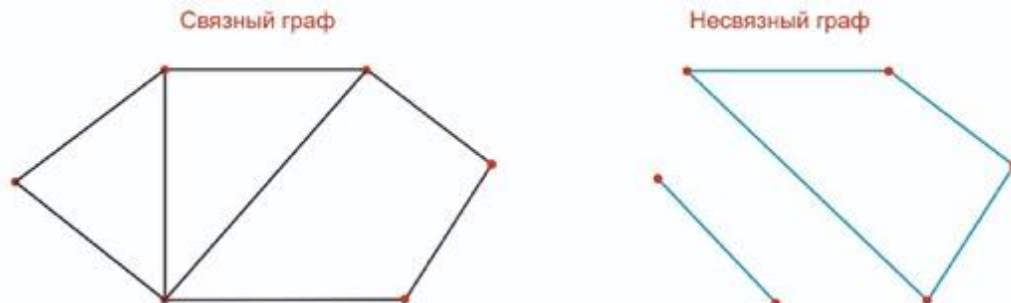


Если в графе степень каждой вершины больше или равна 2, то в нем есть цикл.

Доказательство. Пусть $P = (x_1, x_2, \dots, x_k)$ – путь наибольшей длины в графе. Так как степени всех вершин не меньше 2, то вершина x_k смежна, кроме вершины x_{k-1} , еще с какой-нибудь вершиной u . Вершина u не может отличаться от всех вершин пути P , иначе образовался бы более длинный путь $x_1, x_2, \dots, x_k, u$. Значит, $u = x_i$ при некотором $i < k - 1$. Но тогда $x_i, x_{i+1}, \dots, x_k, x_i$ – цикл.

Граф называется *связным*, если для любых двух его вершин имеется путь, соединяющий эти вершины. Если граф несвязен, то он состоит из нескольких связных подграфов, между которыми нет ребер, они называются *компонентами связности*.

Орграф называется *связным*, если между любыми двумя вершинами имеется соединяющий их неориентированный путь. Он называется *сильно связным*, если из любой вершины в любую другую имеется ориентированный путь.



Эксцентриситет [eccentricity] *вершины* – это наибольшее из расстояний от неё до остальных вершин графа.

Диаметр [diameter] графа – наибольший из эксцентриситетов его вершин.

Радиус [radius] графа – наименьший из эксцентриситетов его вершин.

Центральная вершина [center/central vertex] – вершина графа, эксцентриситет которой равен радиусу графа.

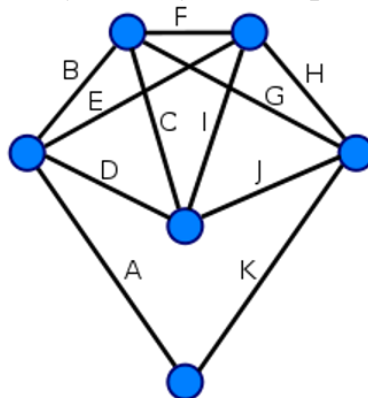
Эйлеровой цепью в неориентированном графе G называется простая цепь, содержащая все ребра графа G . **Эйлеровым циклом** называется замкнутая Эйлерова цепь. Аналогично, **эйлеров путь** в орграфе G – это простой путь, содержащий все дуги графа G . **Эйлеров контур** в орграфе G – это замкнутый эйлеров путь. Граф, в котором существует эйлеров цикл, называется *эйлеровым*.

Эйлеров путь (эйлерова цепь) в графе – это путь, проходящий по всем рёбрам графа и притом только по одному разу.

Эйлеров цикл - эйлеров путь, являющийся циклом, то есть замкнутый путь, проходящий через каждое ребро графа ровно по одному разу.

Полуэйлеров граф – граф, в котором существует эйлеров путь.

Эйлеров граф – граф, в котором существует эйлеров цикл.



Каждая вершина этого графа имеет чётную степень, поэтому этот граф - эйлеров. Обход рёбер в алфавитном порядке даёт эйлеров цикл

Пример 1

Задача:

Разработайте программу на языке C++, реализующую неориентированный граф с использованием стандартных контейнеров STL.

Для представления графа используйте структуру списка смежности, в которой каждая вершина хранится в ассоциативном контейнере и связана со списком всех смежных (соседних) вершин. Программа должна реализовывать обход графа в ширину.

Решение:

```
1  #include <iostream>
2  #include <map>
3  #include <list>
4  #include <queue>
5  #include <set>
6
7  using namespace std;
8
9  class Graph {
10 private:
11     map<string, list<string>> adjList; // СПИСОК СМЕЖНОСТИ
12 public:
13     void addEdge(const string& u, const string& v) {
14         adjList[u].push_back(v);
15         adjList[v].push_back(u);
16     }
17     void printList() const {
18         cout << "\nСписок смежности:\n";
19         for (const auto& pair : adjList) {
20             cout << pair.first << " -> ";
21             for (const auto& ng : pair.second) {
22                 cout << ng << " ";
23             }
24             cout << endl;
25         }
26     }
27
28     void bfs(const string& start) const {
29         set<string> visited;
30         queue<string> q;
31
32         visited.insert(start);
```

```

33     q.push(start);
34     cout << "\nОбход в ширину (BFS), начиная с вершины " << start << ": ";
35     while (!q.empty()) {
36         string current = q.front();
37         q.pop();
38         cout << current << " ";
39
40         for (const auto& ng : adjList.at(current)) {
41             if (visited.find(ng) == visited.end()) {
42                 visited.insert(ng);
43                 q.push(ng);
44             }
45         }
46     }
47     cout << endl;
48 }
49 };
50 int main() {
51     Graph g;
52     int n;
53     cout << "Введите количество рёбер: ";
54     cin >> n;
55     cout << "Введите рёбра:" << endl;
56     for (int i = 0; i < n; ++i) {
57         string u, v;
58         cin >> u >> v;
59         g.addEdge(u, v);
60     }
61
62     g.printList();
63
64     string start;
65     cout << "\nВведите начальную вершину для обхода в ширину: ";
66     cin >> start;
67     g.bfs(start);
68
69     return 0;
70 }

```

Ответ:

Введите количество рёбер:

5

Введите рёбра:

A B

A C

B D

D E

E C

Список смежности:

A → B C

B → A D

C → A E

D → B E

E → D C

Введите начальную вершину для обхода в ширину:

A

Обход в ширину (BFS), начиная с вершины A: A B C D E

Задание 1 (1 балл)

Задача:

Разработайте программу, которая позволяет пользователю выбрать способ задания графа: с помощью матрицы смежности или матрицы инцидентности. Программа должна запросить у пользователя количество вершин и рёбер, после чего предложить ввести соответствующую матрицу. Также необходимо реализовать простой алгоритм обхода графа: поиск в глубину (DFS) или поиск в ширину (BFS) и вывести порядок обхода вершин.

☐ **Замечание:** программа должна определять автоматически является ли граф ориентированным или неориентированным.

Решение:



Ответ:



Задание 2 (2 балла)

Задача:

В некоем королевстве существует сеть дорог между городами, представленная в виде неориентированного графа. Из-за особенностей местного климата некоторые дороги становятся невидимыми: они есть, но их не видно на карте. Тем не менее, жители всё равно могут по ним перемещаться.

Вам известны все города (вершины) и только те дороги, которые остались видимыми (рёбра). К счастью, вы также знаете, что изначально все города были

соединены так, что можно было добраться от любого города до любого другого. Какое минимальное количество дорог необходимо «восстановить», чтобы вновь гарантировать связность всей карты?

Вход:

Вершины: A, B, C, D, E

Видимые рёбра:

A B

C D

Выход:

В графе 3 компоненты связности: {A, B}, {C, D}, {E}

Нужно добавить минимум 2 ребра, чтобы соединить все компоненты.

☐ **Подсказка:**

Количество недостающих рёбер для связности = количество компонент связности – 1.

Решение:



Ответ:



Нахождение кратчайшего пути в графе. Алгоритм Прима

Теоретический материал

Задача о кратчайшем пути в заданный пункт назначения

Задача о кратчайшем пути - задача поиска самого короткого пути (цепи) между двумя точками (вершинами) на графе, в которой минимизируется сумма весов рёбер, составляющих путь.

Задача о кратчайшем пути является одной из важнейших классических задач теории графов.

Пусть G – связный взвешенный граф и пусть v, v' – две вершины графа G . Требуется найти путь с самым маленьким весом, соединяющий вершины v и v' . Вес пути определяется как сумма весов его рёбер.

Идея *метода Дейкстры* состоит в том, чтобы приписывать по очереди каждой вершине u число $L(u)$ -метку, которая представляет собой длину самого короткого пути, обнаруженного между вершинами v и u . Самой исходной вершине v приписывается постоянная нулевая метка. Значения меток $L(u)$ первоначально рассматриваются *временными* и могут быть впоследствии заменены, если будет обнаружен путь между вершинами v и u , имеющий величину длины меньшую, чем назначенную в текущий момент времени

Последовательность шагов алгоритма:

1. Начальному узлу приписывают постоянную нулевую метку.
2. Затем определяют узлы, которые можно достигнуть из начального узла. Им приписывают временные метки, равные длине пути из исходного узла.
3. Выбираем узел с кратчайшим путем и приписываем ему постоянную метку. Для этого необходимо следующее:
 - 3.а Рассмотрим оставшиеся на данном шаге узлы с временной меткой.
 - 3.б Среди временных меток выбираем ту, значение которой минимально и объявляем ее постоянной меткой. Если при этом постоянную метку приписывают конечному узлу, то задача решена. В противном случае переходим на шаг 1.

Алгоритм Прима

В теории графов остовное дерево (spanning tree) - это подмножество рёбер связного неориентированного графа, соединяющее все вершины без образования циклов. Если каждому ребру присвоен вес (например, расстояние, стоимость, время и т.д.), то минимальное остовное дерево (МОД) - это остовное дерево с минимальной суммарной стоимостью рёбер.

Существует несколько эффективных алгоритмов для построения МОД. Наиболее известны: алгоритм Краскала и алгоритм Прима.

Алгоритм Прима строит минимальное остовное дерево, начиная с произвольной вершины и последовательно добавляя в дерево наиболее дешёвое ребро, соединяющее уже включённую вершину с невключённой. Это напоминает стратегию постепенного распространения по графу.

Формальная процедура алгоритма Прима:

1. Выбрать произвольную стартовую вершину v_0 и включить её в остовное дерево.
2. Пока в дереве не будут все вершины:
 - выбрать ребро минимального веса, соединяющее дерево с невключённой вершиной;
 - добавить это ребро и вершину в остовное дерево.

Для реализации используется очередь с приоритетом (например, бинарная куча), что позволяет эффективно находить минимальное ребро.

Асимптотическая сложность алгоритма Прима равна $O(N^2)$. Алгоритм особенно эффективен для плотных графов.

Задание 3 (2 балла)

Задача:

В таблице приведено расстояние между кампусами РТУ МИРЭА

	Пр-т Вернадского, 78	Пр-т Вернадского, 86	Стромынка, 20	Малая Пироговская, 1	5-я улица Соколиной горы, 22	Усачева, 7
Пр-т Вернадского, 78	-	1.97	21.6	10.7	22.3	10.4
Пр-т Вернадского, 86	1.97	-	22.3	11.4	23	11.1

Стромынка, 20	21.6	22.3	-	11.5	5.2	12
Малая Пироговская, 1	10.7	11.4	11.5	-	13.4	0.68
5-я улица Соколиной горы, 22	22.3	23	5.2	13.4	-	13.8
Усачева, 7	10.4	11.1	12	0.68	13.8	-

С помощью алгоритма Прима выделить маршрут с минимальной общей длиной, связывающий все кампусы РТУ МИРЭА

Решение:

Ответ:

Задание 4 (3 балла)

Задача:

Реализуйте представление ориентированного графа согласно структуре Вирта. Обеспечьте ввод графа с клавиатуры (задание списка вершин и рёбер), построение связной структуры в памяти и вывод списка смежности для каждой вершины.

Дополнительно реализуйте простейший алгоритм обхода графа в глубину (DFS), используя разработанную структуру.

Решение:

Ответ:

Задание 5*

Задача:

На карте города изображена сеть дорог, представляющая собой связный неориентированный граф. Каждое ребро графа имеет неотрицательный вес, соответствующий длине дороги. Почтальон должен пройти по всем дорогам хотя бы один раз, вернувшись в начальную точку, при этом общий путь должен быть минимальным по длине. Разработайте программу, которая решает данную задачу.

Входные данные:

Вершины: A, B, C, D

Рёбра:

A B 4

A C 3

B C 2

C D 5

Выходные данные:

Решение задачи китайского почтальона:

A -> C

C -> D

D -> C

C -> B

B -> A

Общая стоимость маршрута: 19

Решение:



Ответ:



Задание 6**

Задача:

Создайте программу, которая позволяет определить маршрут, проходящий через каждый город ровно один раз и возвращающийся в исходную точку, причём суммарная длина (или стоимость) всего пути должна быть минимальной (задача коммивояжера).

Решение:



Ответ:

